



Jour 3 — Intégration Docker et Kubernetes

Exécuter des tâches en containers et sur K8s

- 🎯 **Objectif** : Maîtriser DockerOperator, KubernetesPodOperator et Helm
- 📦 **Livrable** : Déploiement Helm + tâches KPO opérationnelles



Docker



Kubernetes



Helm

Objectifs du Jour 3

Intégration Docker et Kubernetes : déployer et exécuter sur K8s

Objectif 1 - Comprendre DockerOperator

Maîtriser l'exécution de tâches conteneurisées pour isoler les dépendances et standardiser l'exécution.

Objectif 3 - Utiliser KubernetesPodOperator

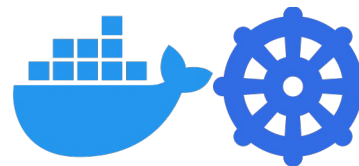
Lancer des pods éphémères sur K8s avec gestion fine des ressources, secrets et volumes.

Objectif 2 - Comparer les Executors

Analyser CeleryExecutor vs KubernetesExecutor pour choisir l'architecture adaptée à vos besoins.

Objectif 4 - Déployer via Helm

Installer et configurer Airflow sur Kubernetes en utilisant le Helm chart officiel.



Rappels Docker pour Airflow

Concepts fondamentaux pour containeriser vos workflows



Images : Base Python + Providers

Image de base standard `apache/airflow:X.Y.Z-python3.N`.
Nécessité de créer une image personnalisée pour ajouter vos dépendances Python spécifiques et providers.



Volumes : Persistence et Code

Monter les répertoires `/dags`, `/logs`, et `/plugins` permet de mettre à jour le code sans reconstruire l'image et de conserver les logs d'exécution.



Réseau et Variables

Configuration via variables d'environnement (`AIRFLOW\_\*`).
Accès réseau crucial pour joindre la Metadata DB, Redis (Celery), et les services externes (S3, APIs).



```
FROM apache/airflow:2.8.0
USER root

RUN apt-get update && apt-get install -y gcc

USER airflow

RUN pip install --no-cache-dir \
    apache-airflow-providers-docker \
    pandas dbt-core
```

Reproductibilité Garantie

Un environnement identique du développement à la production

DockerOperator : Cas d'usage

Pourquoi utiliser DockerOperator ?

-  **Encapsuler** des scripts "out-of-tree" qui ne sont pas en Python natif (R, Go, Rust, etc.).
-  **Isoler** les dépendances lourdes pour éviter les conflits dans l'environnement Airflow principal.
-  Garantir la **reproductibilité** de l'exécution grâce aux images Docker immutables.
-  Interagir avec les **réseaux et volumes** Docker directement depuis le DAG.

```
python - docker_operator_example.py

from airflow import DAG
from airflow.providers.docker.operators.docker import DockerOperator
from docker.types import Mount
from datetime import datetime

# Définition de la tâche Docker
run_data_job = DockerOperator(
    task_id='run_data_processing',
    image='myrepo/data-job:1.0',
    container_name='task_processing_container',
    api_version='auto',
    auto_remove='force', # Nettoie après exécution,
    command='python job.py --date {{ ds }}',
    docker_url='unix://var/run/docker.sock',
    network_mode='bridge',
    environment={
        'DB_HOST': 'postgres-db',
        'API_KEY': '{{ var.value.api_key }}'
    },
    mounts=[
        Mount(source='/local/data', target='/app/data', type='bind')
    ]
)
```

Bonnes pratiques Docker

Recommandations essentielles pour l'intégration sécurisée et performante de conteneurs.



Images minimales

Utilisez des images légères (**distroless** ou **alpine** quand possible) pour réduire la surface d'attaque et accélérer le démarrage des tâches.



Pin des versions & digest SHA

Fixez toujours les versions de vos images et utilisez les **digests SHA**. Cela garantit une reproductibilité totale en évitant les tags flottants comme `latest`.



Sécurisation des secrets

Ne passez **jamais de secrets en clair** dans le code. Utilisez les connexions Airflow ou injectez-les dynamiquement via les variables d'environnement (ENV).

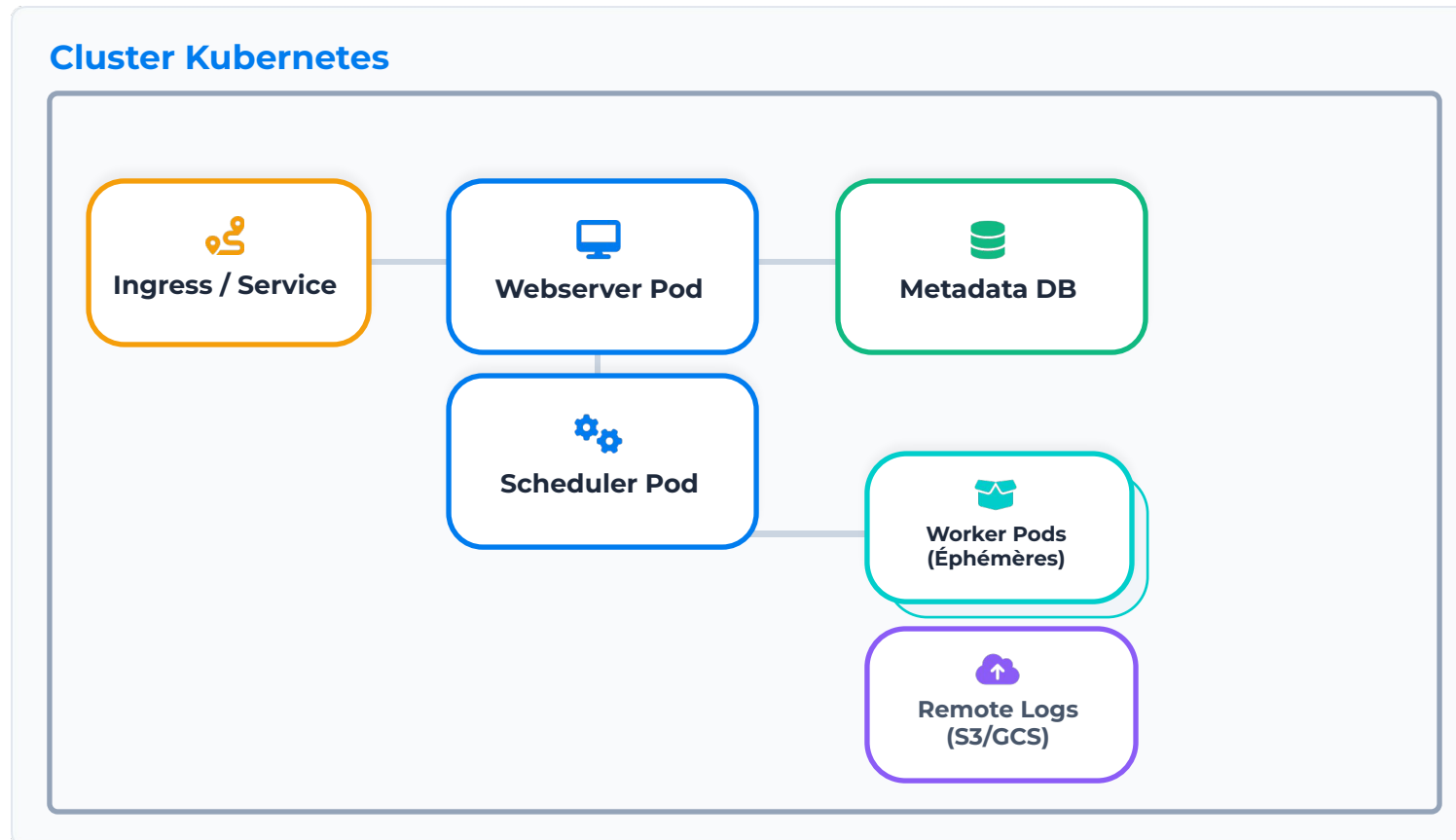


Nettoyer volumes et artefacts

Pensez à **nettoyer les volumes** montés et les artefacts temporaires générés par les conteneurs pour éviter toute saturation de l'espace disque sur les workers.

Airflow sur Kubernetes: architecture

Déploiement cloud-native avec pods éphémères



Composants de base

Webserver, Scheduler et DB tournent dans des pods persistants (Deployments/StatefulSets).

Accès UI via Ingress ou Service Kubernetes.

Exécution dynamique

Avec KubernetesExecutor, le scheduler lance un **nouveau pod pour chaque tâche**.

Isolation parfaite : chaque tâche a ses propres dépendances/ressources.

Stockage externe

Les pods workers étant éphémères, les logs **doivent** être envoyés vers un stockage distant (ex: S3, GCS) pour être consultables via l'UI.

CeleryExecutor vs KubernetesExecutor

Comparaison des deux approches principales pour l'exécution distribuée en production.

Critère	CeleryExecutor	KubernetesExecutor
Architecture	Nécessite un broker (Redis/RabbitMQ) et un pool de workers persistants.	Pas de broker. Utilise l'API Kubernetes pour lancer des pods éphémères à la volée.
Isolation	Partagée. Les tâches s'exécutent dans le même environnement (le worker).	Forte. Chaque tâche s'exécute dans son propre pod avec sa propre image Docker.
Autoscaling	Nécessite de scaler les workers (ex: via KEDA). Difficile de scaler à zéro.	Natif et dynamique. Le cluster K8s gère les pods. Scale à zéro sans tâche en attente.
Latence au démarrage	Très faible. Les workers tournent en permanence et dépilent instantanément.	Plus élevée. Temps d'allocation du pod et de récupération (pull) de l'image.
Coûts / Ressources	Fixes. Les workers consomment des ressources même s'ils sont inactifs.	Variables. Les ressources sont allouées et facturées uniquement à l'exécution.

KubernetesPodOperator : principes

Concept fondamental

-  Exécute un **container Docker** dans un pod Kubernetes dédié
-  **Isolation totale** : dépendances et environnement d'exécution isolés
-  Permet d'exécuter des tâches dans **différents langages** (Python, R, Go, etc.)
-  Les pods sont **éphémères** (créés pour la tâche, détruits après exécution)



Configuration et paramètres clés

-  **Image** : l'image Docker à exécuter (ex: `python:3.9-slim`)
-  **Cmds / Args** : la commande à exécuter au démarrage du container
-  **Resources** : limites et requêtes CPU/Mémoire pour garantir la stabilité
-  **Mounts** : volumes (ConfigMaps), secrets et ServiceAccounts Kubernetes

KPO : Exemple minimal

Configuration essentielle

- 1 Spécifier l'**image** Docker à utiliser (ex: `python:3.11-slim`).
- 2 Définir les commandes (**cmds**) et arguments (**arguments**) à exécuter.
- 3 Allouer les **ressources** nécessaires (CPU/Mémoire) pour éviter les OOM kills.
- 4 Indiquer le **namespace** Kubernetes où le pod sera déployé.

```
python - KPO_example.py

from airflow import DAG
from airflow.providers.cncf.kubernetes.operators.pod import
KubernetesPodOperator
from kubernetes.client import models as k8s

# Définition des ressources K8s
compute_resources = k8s.V1ResourceRequirements(
    requests={"cpu": "500m", "memory": "512Mi"},
    limits={"cpu": "1000m", "memory": "1Gi"}
)

# Instanciation de l'opérateur
kpo_task = KubernetesPodOperator(
    task_id="tache_k8s_minimale",
    name="pod-minimal",
    namespace="airflow-tasks",
    image="python:3.11-slim",
    cmds=["python", "-c"],
    arguments=["print('Exécution réussie depuis K8s !)"]],
    container_resources=compute_resources,
    is_delete_operator_pod=True,
    get_logs=True
)
```

Helm Chart Airflow : composants

Déployer Airflow de manière déclarative et scalable sur Kubernetes



Templates (Ressources K8s)

Définitions des Déploiements (Webserver, Scheduler), StatefulSets (Workers), Services, Ingress, ConfigMaps, Secrets, RBAC.



Values (Configuration)

Fichier `values.yaml` pour paramétrer : l'executor (Celery/K8s), variables d'environnement, connexions DB externes, persistance des volumes (logs, dags).



Addons & Dépendances

Intégration de composants optionnels : PostgreSQL/MySQL (DB metadata), Redis (Broker Celery), Flower (Monitoring Celery), StatsD (Métriques).



values.yaml



Helm
Chart



Cluster
K8s

Déploiement Unifié

Un seul outil pour gérer l'installation, la mise à jour (`helm upgrade`) et la configuration de l'ensemble de l'architecture Airflow.

Déploiement avec Helm : étapes

Guide pas à pas pour déployer Apache Airflow sur un cluster Kubernetes via Helm.

Ajouter repo & Installer

Ajoutez le repository officiel et lancez l'installation avec votre `values.yaml`.

```
helm repo add apache-airflow \
  https://airflow.apache.org

helm install airflow \
  apache-airflow/airflow \
  -f values.yaml
```

Configuration K8s

Configurez RBAC/SA, les Secrets et l'Ingress pour l'exposition de l'UI.



RBAC & Sécurité

Valider le déploiement

Vérifiez l'état des pods (Running) et accédez à l'interface UI (port-forward).



```
kubectl get pods -n airflow
```

Configurer executor et ressources

Points de configuration

- 1 **Executor** : Activer `KubernetesExecutor` pour isoler chaque tâche dans son propre pod éphémère.
- 2 **Logs distants** : Indispensable sur K8s ! Les pods étant détruits après exécution, les logs doivent être persistés sur S3, GCS ou Azure Blob.
- 3 **Concurrency** : Ajuster `parallelism` (limite globale du cluster) et la concurrence par DAG selon la capacité du cluster.
- 4 **Ressources** : Définir le `kubernetesPodTemplate` pour allouer des CPU/RAM par défaut aux workers.

```
yaml - values.yaml

# values.yaml - Configuration de l'environnement
executor: "KubernetesExecutor"

config:
  logging:
    remote_logging: 'True'
    remote_base_log_folder: 's3://my-airflow-logs/prod'
    remote_log_conn_id: 'aws_default'
  core:
    parallelism: '100'
    max_active_tasks_per_dag: '16'

# Template de base pour les pods d'exécution
kubernetesPodTemplate:
  resources:
    requests:
      cpu: "500m"
      memory: "512Mi"
    limits:
      cpu: "1"
      memory: "1Gi"
```

Autoscaling et quotas sur Kubernetes

Bonnes pratiques pour contrôler l'échelle et la consommation des ressources.



HPA / VPA pour Airflow

Utilisez un **Horizontal Pod Autoscaler** ou **Vertical Pod Autoscaler** pour le Webserver et le Scheduler afin d'absorber les pics de charge ponctuellement.



Quota de namespaces & PDB

Définissez des limites matérielles par namespace (ResourceQuotas) et protégez les composants core via un **PodDisruptionBudget (PDB)** lors des évictions de nœuds.



Tuning de concurrence

Contrôlez l'explosion des pods en paramétrant `parallelism`, `dag_concurrency` et `max_active_runs_per_dag` dans ``airflow.cfg`` ou Helm values.



Observabilité Kubernetes

Utilisez **kube-state-metrics** combiné à Prometheus/Grafana pour surveiller l'état des pods travailleurs et anticiper les goulots d'étranglement du cluster K8s.

TP Jour 3 : DockerOperator

Mise en pratique : Exécuter des tâches encapsulées dans des conteneurs Docker

Consignes de l'exercice

- 1 **Créer un DAG** contenant une tâche qui utilise le `DockerOperator` pour exécuter une image Docker de votre choix (ex: `python:3.9-slim`).
- 2 **Paramétrer l'opérateur :**
Définir la commande à exécuter (ex: `python -c "print('Hello from Docker!')"`).
Passer des variables d'environnement via le paramètre `environment`.
Monter un volume local dans le conteneur via le paramètre `mounts`.
- 3 **Exécuter et analyser :** Lancez le DAG, vérifiez que le conteneur démarre correctement et consultez les logs publiés par le script.

Objectifs Bonus

- ✓ Créer un script Python plus complexe qui génère un fichier de résultat dans le volume monté.
- ✓ Récupérer ce fichier dans une tâche suivante (ex: avec un `PythonOperator`) ou l'envoyer vers S3.

Conseils & Astuces

- Assurez-vous que l'utilisateur Airflow a les droits d'accès au socket Docker (`/var/run/docker.sock`).
- Utilisez `auto_remove='force'` pour nettoyer le conteneur après son exécution et éviter de saturer le disque.

TP : KubernetesPodOperator

Mise en pratique : Exécution de tâches dans des pods éphémères sur Kubernetes

Consignes de l'exercice

- 1 **Déployer un job Python simple sur K8s** en utilisant `KubernetesPodOperator`. Utilisez une image Python standard (ex: `python:3.9-slim`) et une commande simple (ex: `python -c "print('Hello K8s')"`).
- 2 **Limiter les ressources** du pod :
Définissez des requests et limits pour le CPU (ex: 100m/200m).
Définissez des requests et limits pour la mémoire (ex: 128Mi/256Mi).
- 3 **Collecter les logs** : Assurez-vous que l'option `get_logs=True` est activée et vérifiez la sortie dans l'interface UI d'Airflow après l'exécution.

Objectifs Bonus

- ✓ Utiliser un secret monté en volume ou en variable d'environnement pour passer des informations sensibles au pod.
- ✓ Configurez `is_delete_operator_pod=True` pour nettoyer le pod après le succès.

Conseils & Astuces

- Pour tester localement, assurez-vous d'avoir configuré le contexte Kubernetes (`~/.kube/config` ou la connexion `kubernetes_default` dans Airflow).
- Pensez à vérifier l'état du pod via `kubectl get pods -n <votre-namespace>` si la tâche semble bloquée.

Checkpoint Jour 3

Testez vos connaissances sur les concepts d'intégration Docker et Kubernetes :

1

Quel est l'atout principal du K8sExecutor par rapport aux autres ?

?

2

Dans quels cas précis devriez-vous préférer CeleryExecutor ?

?

3

Citez 3 champs essentiels à configurer pour un KPO.

?

4

Comment et où stocker de manière fiable les logs en production sur K8s ?

?

Récap — Jour 3

Points clés à retenir de cette troisième journée sur Kubernetes et Docker :



Tâches isolées et reproductibles via l'utilisation du **DockerOperator**



Maîtrise du **KubernetesExecutor** et du **KubernetesPodOperator**



Déploiement d'Airflow opérationnel en production avec **Helm**



L'essentiel : K8s apporte isolation, scalabilité et flexibilité, posant les bases pour des pipelines Big Data robustes.